

Labels, Arrows, and Memory

The one model under reference, mutation, immutability, copying, undefined, and leaks

October 2026

The model.

A variable is a label. A label holds an arrow. The arrow points at a value that lives somewhere in memory. Assignment moves the arrow; it never moves the value. This is the same model as the Python notes (“a variable is a label bound to an object, not a box”), and every rule below is a consequence of it.

label	arrow	memory	
-----	-----	-----	
a	→	[3, 1]	one array, at some address
b	→	(same)	a second label on the same address
s	→	"abc"	a string, at another address
n	→	7	a number
x	→	(nothing)	undefined: an arrow to nowhere

1. Assignment moves the arrow

`b = a` copies the **arrow**, not the thing it points at. Afterwards two labels point at one value.

```
const a = { n: 1 };
const b = a;           // b's arrow now points where a's does
b.n = 2;               // follow b's arrow, edit the object there
a.n                    // 2: a's arrow points at the same object
```

Say: “`b` is set to `a`” means “`b` now points at whatever `a` points at.” Nothing was duplicated.

2. Mutation edits the value; reassignment re-points the arrow

Two different operations that look similar on the page.

Operation	What moves	Example	Who notices
mutation	the contents at the address	<code>a.push(1)</code> , <code>a.n = 2</code> , <code>a.sort()</code> , <code>delete a.k</code>	every label pointing there
reassignment	the label's arrow	<code>a = [1]</code> , <code>a = null</code> , <code>a = a.filter(...)</code>	only this label

```
let a = [1]; const b = a;
a.push(2);      // mutation: a and b both see [1, 2]
a = [9];        // reassignment: a's arrow moves; b still points at [1, 2]
```

`const` forbids the second row only. `const a = []`; `a.push(1)` is legal; `a = []` is a `TypeError`. `const` locks the arrow, not the contents.

3. Immutable means the contents cannot be edited

Strings, numbers, booleans, `null`, `undefined`, `bigint`, `symbol` are **immutable**: no operation edits them at their address. Every “change” builds a new value at a new address and hands back an arrow to it, which you must store.

```
let s = "abc";
s.toUpperCase(); // builds "ABC" somewhere; nobody stores the arrow; s still -> "abc"
s = s.toUpperCase(); // s's arrow re-pointed at "ABC"; "abc" is now unreferenced
s[0] = "x";         // silently ignored: the contents cannot be edited
```

Because nothing can edit an immutable value in place, two labels pointing at the same string can never surprise each other. That is why primitives are described as “copied”: the observable behavior is identical to copying, and you can reason with that rule even though the engine may share the bytes.

	Immutable (string, number, boolean)	Mutable (array, object, Map, Set, class instance)
Can the contents be edited?	no	yes

	Immutable (string, number, boolean)	Mutable (array, object, Map, Set, class instance)
What do methods return?	always a new value	some edit in place (<code>push</code> , <code>sort</code>), some return new (<code>map</code> , <code>filter</code>)
Must I store the result?	always	only for the new-value methods
Can two labels surprise each other?	never	yes, whenever they share an address
When do I copy on purpose?	never needed	when one label must change without the other seeing it

Python parallel: `str`, `int`, `tuple` immutable; `list`, `dict`, `set` mutable. Same rules.

4. Copying makes a new address on purpose

A copy is a new value at a new address with the same contents, so a label pointing at the copy can mutate it without touching the original.

Written	Makes	Depth
<code>[...a]</code> , <code>a.slice()</code> , <code>Array.from(a)</code>	new array	shallow
<code>{ ...o }</code> , <code>Object.assign({}, o)</code>	new object	shallow
<code>new Map(m)</code> , <code>new Set(s)</code>	new collection	shallow
<code>a.filter(...)</code> , <code>a.map(...)</code>	new array	shallow (the elements are the same objects)
<code>structuredClone(o)</code>	new everything	deep
<code>JSON.parse(JSON.stringify(o))</code>	new everything, but drops <code>undefined</code> , <code>Date</code> , <code>Map</code> , <code>Set</code> , functions	deep-ish

Shallow

means one level: the outer array or object is new, but the arrows inside it still point at the same inner objects.

```
const o = { inner: { x: 1 } };
const c = { ...o }; // c is a new object; c.inner is the SAME inner object as o.inner
c.inner.x = 9;      // o.inner.x is 9 too
const d = structuredClone(o);
d.inner.x = 5;      // o.inner.x unchanged
```

`filter` is the trap: it copies the array but not the objects in it, so `orders.filter(...).forEach((o) => o.status = "paid")` mutates the caller's orders.

5. `undefined` and `null` are arrows to nothing

A label can hold an arrow that points at no object. `undefined` is “never given anything to point at”; `null` is “deliberately pointed at nothing”.

```
let x; // x -> undefined
const o: { a?: number } = {};
o.a // undefined: no arrow stored under key a
users.find(...) // undefined when nothing matched
map.get("missing") // undefined
arr[arr.length] // undefined: one past the end
```

Following an arrow to nothing is the crash: `x.name` means “go where `x` points and read `name`”, and there is no “there”. `TypeError: Cannot read properties of undefined (reading 'name')`.

`?.` means “follow the arrow only if it points somewhere; otherwise hand back `undefined`”. `??` means “if the arrow points at nothing, use this instead”. `!` means “I promise this arrow points somewhere” and checks nothing.

6. Garbage collection and leaks

Memory you no longer point at is reclaimed automatically. When the last arrow to a value is moved away or its label goes out of scope, the value is freed. You never free anything by hand.

A **leak** is therefore always the same thing: *an arrow that should have been dropped is still held*. The value cannot be collected because something still points at it.

Leak	The arrow that keeps it alive
a <code>Map</code> cache with no eviction	the map's own entries
a per-key array that is never trimmed	the array inside the map
<code>setInterval</code> never cleared	the timer holds the callback, the callback holds its closure
an event listener never removed	the emitter holds the listener
a module-level array that only grows	the module
a closure that captured a large object	the function that is still referenced

Say: “What arrow is keeping this alive, and who drops it?” If nobody drops it, it grows forever.

7. Function calls pass arrows

Arguments are passed the same way assignment works: the parameter is a new label pointing at the same value. Mutating a parameter mutates the caller’s object; reassigning a parameter changes only the local label.

```
function f(xs: number[]) {
  xs.push(4);           // caller's array now has 4: same address
  xs = [];              // only f's label moved; caller unaffected
}
```

Default parameters are evaluated fresh on every call, so `function g(xs = [])` gives a new empty array each time; the Python shared-default trap does not exist here.

8. Equality compares arrows for objects, values for primitives

`===` on two objects asks “same address?”, never “same contents?”. `===` on two primitives asks “same value?”.

```
{ a: 1 } === { a: 1 }           // false: two addresses
const p = { a: 1 }; const q = p; p === q // true: one address
"a" === "a"                     // true: same value
[1] === [1]                     // false
arr.includes({ a: 1 })          // false unless that exact object is in arr
set.has({ a: 1 })               // false for a fresh literal
```

To compare contents: field by field, `JSON.stringify` both (key-order sensitive), or a deep-equal helper. Dates: `.getTime()`.

The model applied to the review questions

Review question	In arrow terms
Who else is holding this? (mutation)	how many labels point at this address?
Did the result get caught? (new-value methods)	was the arrow to the new value stored anywhere?
Can this be <code>undefined</code> here?	does this arrow point at something before I follow it?
Does this grow forever? (leaks)	what arrow is keeping this alive, and who drops it?
Is this a copy or an alias?	same address or new address, and how deep?
Are these equal?	same address (objects) or same value (primitives)?

One sentence to keep. Assignment moves arrows; mutation edits what an arrow points at; immutable values cannot be edited so every change is a new address; a copy is a new address made on purpose; `undefined` is an arrow to nowhere; a leak is an arrow nobody dropped.